

Learning as Amortized Deduction: Ceilings, Efficiency, and Size Transfer in Binary Linear Systems

학습은 amortized deduction이다: binary linear system에서의 상한, 효율, 그리고 크기 전이
neuroMCTS project · 2026년 9월 · 한국어판

초록

Binary linear system(BLS)은 $A \in \{0,1\}^{m \times n}$ 에 대해 $Ax=b$ 를 만족하는 $x \in \{0,1\}^n$ 를 찾는 문제로, 레이저 기반 비파괴검사에서 정수 투영값으로부터 이진 영상을 복원하는 과제에서 나온다. feasibility 판정은 NP-complete이며, 본 연구가 다루는 market-split 계열에서는 classical solver가 지금까지 시도한 모든 학습 기반 방법을 압도한다. 11개 사이클에 걸쳐 12개의 학습 컴포넌트를 시험했고 11개가 null result였는데, 이는 "입력에 학습 가능한 signal이 없다"는 해석으로 이어져 있었다. 본 보고서는 그 해석이 틀렸음을 보이고, 해석 대신 측정을 제시한다.

생성기가 해 x^* 를 심고 $b=Ax^*$ 를 공개하므로, solution set S 의 모든 원소가 동일한 b 를 만든다. 따라서 posterior는 S 위에 uniform이고 per-variable marginal p_i 를 전수열거로 정확히 계산할 수 있다. 이로부터 어떤 predictor도 넘을 수 없는 Bayes ceiling이 얻어진다. 네 가지 결과가 따라온다. 첫째, hard 계열에서 ceiling은 per-variable 69.9%(가장 확실하는 변수 3개 기준 93.1%)이고, 작은 단일 head bipartite GNN이 그 1.9%p 이내에 도달한다 — signal은 실재하고, 유한하며, 사실상 소진됐다. 반면 기존의 multi-head network는 단순 LP rounding 수준에 머물렀다(60.7% vs 60.6%). 둘째, n 을 고정하고 m 을 바꾸면 ceiling 자체가 실험 변수가 되며, 세 구간이 갈린다: ceiling이 낮은 곳에서는 모델이 이미 포화하고, 높은 곳에서는 도달이 NP-hard이며(18~20%p 잔차), 더 높은 곳에서는 LP relaxation만으로 충분하다. 셋째, partial assignment로 conditioning하는 것은 instance를 축소하는 것과 동일하므로 같은 architecture가 search tree 내부에서 예측할 수 있다. conditional ceiling은 root의 70.5%에서 depth 8의 93.4%로 상승하며, 잔차를 분해하면 전부 prefix가 논리적으로 결정하는 변수 위에 있는데 그중 92~99%를 unit propagation이 탐지하지 못한다. LP-probing은 62~79%를 회수하지만 변수당 LP 한 번을 지불하고, network는 단일 forward pass 1.6ms에 전 변수를 답하며 적중률은 같거나 높다(약 20배 저렴). 따라서 학습의 역할은 비싼 deduction의 amortization이지 접근 불가능한 정보의 발견이 아니다. 넷째, branching guidance로 쓰면 21×60 에서 LP guidance 대비 median 2.71배 가속(sign test $p=0.019$), node 수 6.2배 감소, 600초 예산 내 해결율 90.0%→100%를 얻는다. lattice reduction이 먼저 도는 실제 cascade에서는 그 단계의 coverage가 $n=25$ 의 30/30에서 $n=60$ 의 16/30으로 떨어지며, guidance가 end-to-end 해결율을 93.3%→100%로 끌어올린다. 크기 간 $|S|$ 를 통제하면 — 이전에는 10배까지 어긋나 있었다 — transfer failure로 오독됐던 artifact가 사라진다: 10×25 에서만 학습한 모델이 21×60 에서 목표 크기로 학습한 모델과 통계적으로 구별되지 않는다($15/30$ 에서 더 빠르고 총 시간 차 0.4%).

1. 서론

1.1 문제와 배경

$A \in \{0,1\}^{m \times n}$, $b \in \mathbb{Z}^m$ 이 주어질 때 binary linear system(BLS)은 $Ax^*=b$ 를 만족하는 $x^* \in \{0,1\}^n$ 를 찾는 문제이며, feasible set은 $S=\{x \in \{0,1\}^n : Ax=b\}$ 이다. 각 row는 cardinality constraint로, 지정된 셀 부분집합 중 몇 개가 채워져 있는지를 말한다.

응용 배경은 레이저 기반 비파괴검사다. 시편을 m 개 방향으로 스캔하면 각 스캔은 경로상 흡수 셀의 정수 개수를 반환하고, 과제는 이진 점유 영상 x 를 복원하는 것이다. 이 설정의 두 성질이 이후 모든 것을 규정한다. 첫째, S 가 동일한 측정치와 부합하는 여러 영상을 담을 수 있으므로 x^* 의 uniqueness를 가정하지 않는다; S 의 원소 아무거나 찾으면 물리적 문제는 해결된다. 둘째, 측정 노이즈(먼지, 캘리브레이션 편차)가 b 를 교란해 $S=\emptyset$ 이 될 수 있으며, infeasibility를 보고하는 것은 해를 반환하는 것만큼 운용상 중요하다.

$S=\emptyset$ 판정은 NP-complete이고, 본 연구의 인스턴스는 market-split 문제에서 영감을 얻은 고난도 영역에서 의도적으로 추출된다 (§ 4.1).

1.2 본 보고서가 답하는 질문

본 프로젝트는 11개 개발 사이클 동안 symbolic solving pipeline에 학습 컴포넌트를 붙여왔다: AlphaZero 방식 MCTS policy, imitation 기반 branching policy, lattice restart scheduler, feasibility classifier 10종, permutation policy, variable-fixing policy. 12개 중 11개가 null result였다. 통용되던 해석은 "(A,b)에 network가 쓸 수 있는 signal이 없다"는 것이었다.

그 해석은 한 번도 검증된 적이 없다. 이것은 입력의 정보량에 대한 주장이고, 정보량은 측정 가능하다. 본 보고서의 핵심 관찰은 이 인스턴스 계열에서 그것이 정확히 측정 가능하다는 것이다:

생성기는 x^* 를 샘플링하고 $b=Ax^*$ 를 공개한다. S의 모든 x 가 동일한 b 를 만들었을 것이므로, 입력의 그 무엇도 이들을 구분하지 못한다. 따라서 "어느 해가 심어졌는가"에 대한 posterior는 S 위에 uniform이고, Bayes-optimal한 per-variable 예측은 $p_j = \Pr[x_j=1 \mid A, b] = |\{x \in S : x_j=1\}| / |S|$ 의 argmax이며, $\max(p_j, 1-p_j)$ 비율로 맞는다. 즉 S를 열거하면 어떤 predictor든 달성 가능한 상한이 정확히 나온다.

열거는 constraint solver로 $n \leq 60$ 에서 실행 가능하므로, ceiling은 이론적 양이 아니라 실험적 양이 된다. 본 보고서의 모든 내용이 이것을 측정할 수 있다는 사실에서 따라 나온다.

1.3 기여

(1) 측정된 information ceiling. hard 계열의 Bayes-optimal per-variable 정확도는 69.9%다 (§ 4.2). 작은 단일 head network가 68.6%로 1.9%p 이내에 도달하는 반면, 기존 multi-head network는 60.7%로 LP rounding(60.6%)과 구별되지 않는다. signal은 실재하고, 작으며, 이제 사실상 소진됐다 — 과거의 null result들은 정보의 부재만큼이나 model과 task의 mismatch를 반영한 것이다.

(2) 통제 변수로서의 ceiling. n 을 고정하고 m 을 올리면 $|S|$ 가 줄고 ceiling이 70.5%에서 100%로 오른다. 이를 sweep하면 세 구간이 드러나며, 학습이 큰 격차로 이기는 지점이 없음을 보인다: ceiling이 낮으면 이미 도달했고, 높으면 도달이 NP-hard이며, 더 높으면 LP relaxation으로 충분하다 (§ 4.3).

(3) amortized deduction으로서의 학습. partial assignment로 conditioning하는 것은 instance 축소와 동일하므로 같은 architecture가 tree 내부에서 예측한다. conditional ceiling은 depth 8에서 93.4%에 이르고, 잔차는 전적으로 prefix가 논리적으로 강제하는 변수 위에 있는데 unit propagation은 그중 0.9~7.5%만 탐지한다. LP-probing은 변수당 LP 한 번(노드당 약 33ms)으로 62~79%를 회수하지만, network는 1.6ms에 전 변수를 답하며 적응률은 같거나 높다 — 19~23배의 비용 우위다 (§ 4.4~4.5).

(4) 종단 성능과 $|S|$ 통제 하의 transfer. branching guidance로서 21×60 에서 LP guidance 대비 2.71배 빠르고($p=0.019$), node를 6.2배 적게 쓰며, 해결율을 90.0%에서 100%로 올린다. 실제 cascade에서는 93.3%를 100%로 올린다. $|S|$ 를 크기 간에 맞추면 10×25 에서만 학습한 모델이 목표 크기로 학습한 모델과 통계적으로 구별되지 않는다 (§ 4.6~4.7).

(5) 원인이 규명된 negative result 지도. 12개 학습 컴포넌트를 각각의 결과 메커니즘과 함께 정리했으며, 측정으로 바로잡은 우리 자신의 오독 2건도 포함한다 (§ 4.8).

2. 배경 및 선행연구

이 절은 자기완결적으로 썼다. 인용 문헌을 보지 않은 독자도 이후 내용을 따라올 수 있도록 하기 위함이다.

2.1 이 인스턴스가 어려운 이유

일반 0/1 행렬 A에 대해 $S \neq \emptyset$ 판정은 NP-complete이므로 worst-case hardness는 논점이 아니다. 실질적으로 중요한 것은 전형적인 인스턴스가 어려운가이고, 그것은 생성 방식에 달려 있다.

본 연구의 계열은 Cornuéjols와 Dawande의 market-split 구성을 따른다. 원본에서는 m명의 agent가 예산을 갖고 n개 품목을 정확히 예산에 맞게 나눠야 하며, $n \approx 10(m-1)$ 일 때 크기가 작은데도 branch-and-bound에 악명 높게 어렵다. 이유는 integrality gap이다: linear relaxation $S_{LP} = \{x \in [0,1]^n : Ax=b\}$ 가 크고 조밀한 polytope이며 그 vertex가 대부분 fractional이다. branch-and-bound는 relaxation이 어떤 변수를 고정해야 하는지 거의 알려주지 않기 때문에 지수적으로 많은 node를 탐색해야 한다. 본 연구는 같은 메커니즘을 의도적으로 이용한다 — 생성기가 vertex spread(무작위 objective 방향에서 얻은 LP vertex들의 평균 쌍거리)를 최대화해 S_{LP} 를 키우고 LP signal을 설계 단계에서 악화시킨다 (§ 4.1).

2.2 Classical solver, 그리고 CP-SAT가 강한 이유

Mixed-integer programming (MIP). Gurobi와 SCIP는 LP relaxation 위의 branch-and-bound로 풀며 presolve와 cutting plane으로 강화한다. 이들의 지렛대는 bounding이다: relaxation이 infeasible한 node는 잘라낸다. relaxation이 무정보할 때 — 정확히 market-split 영역 — 그 지렛대가 약해진다.

Clause learning을 갖춘 constraint programming (CP-SAT). Google OR-Tools의 CP-SAT는 우리 측정에서 일관되게 가장 강한 baseline이며, 그 이유를 정확히 말해줄 필요가 있다. 본 보고서의 논지가 그 이유에서 나오기 때문이다. CP-SAT는 단순 backtracking search가 아니라 다음을 결합한다: constraint propagation(partial assignment에서 강제되는 값을 추측이 아니라 연역한다), conflict-driven clause learning(CDCL)(모순에 도달하면 implication graph를 분석해 어디서나 유효한 새 clause를 유도하고 남은 공간의 지수적으로 큰 영역을 처낸다), lazy clause generation(propagation을 clause 형태로 설명해 CP 수준 추론이 SAT 수준 학습에 공급되게 한다), 그리고 clause를 유지하는 restart와 cutting plane을 공급하는 LP relaxation이다.

핵심은 정보의 출처다. one-shot predictor는 (A,b)에 있는 것만 소비한다. search는 내려가면서 정보를 만들어낸다: $\Pr[x_j=1 \mid A,b]$ 가 정확히 0.5여도 $\Pr[x_j=1 \mid A,b, x_1=1, x_5=0]$ 은 0이나 1일 수 있다. root에서 보이지 않던 구조가 commitment 이후에 연역 가능해진다. CP-SAT가 root marginal이 무정보한 인스턴스를 밀리초에 푸는 이유가 이것이고, § 4.4가 root가 아니라 tree 내부를 보는 이유이기도 하다.

clause learning이 propagation 단독이 아니라 실제 작동 요인인지 확인하기 위해, CDCL이 없는 순수 CP solver(OR-Tools pywrapcp)를 동일 인스턴스에 돌렸다: 10×25 에서는 12/12를 풀지만 20×50 에서는 0/12이며, 같은 지점에서 CP-SAT는 100%를 푼다.

Lattice reduction. 대안적 접근은 문제를 lattice에 embedding하고 짧은 vector를 찾는 것이다. LLL은 다항시간에 reduced basis를 만들고, BKZ는 block size β 로 시간과 basis 품질을 교환하며 이를 일반화한다. Aardal, Hurkens, Lenstra(AHL)는 market-split system의 0/1 해가 적절히 구성된 lattice의 짧은 vector로 직접 나타나는 embedding을 제시했다. 따라서 그것을 찾는 reduction은 탐색 없이 검증된 해를 산출한다. 우리 pipeline에서 AHL은 단일 요소로는 가장 생산적이며, block size는 인스턴스가 커질수록 줄여야 한다: block 40은 $n=100$ 에서 시도당 200초를 넘지만 block 20은 2.4초에 더 나은 coverage를 낸다.

2.3 조합최적화를 위한 학습

Bengio 등(2021)은 이 분야를 개관하면서 본 보고서가 물려받는 표현을 쓴다: 최신 solver는 "계산하기에 너무 비싸거나 수학적으로 잘 정의되지 않은 결정에 대해 수작업 heuristic에 의존"하며, 학습이 그 결정을 더 낮게 만들 후보라는 것이다. 작동하는 표현은 too expensive to compute이다. 이는 그 결정이 원리적으로는 계산 가능함을 전제하고 비용만을 묻는다 — 정확히 § 4.5가 측정하는 상황이며, 입력에 없는 정보를 network에게 요구하는 상황과는 다르다.

학습이 성공한 곳. 가장 분명한 성공은 비싼 expert의 imitation이다. strong branching은 두 자식의 relaxation을 잠정적으로 풀어 branching 후보에 점수를 매기며, 작은 tree를 만들지만 매 node에서 돌리기에 너무 느리다. Gasse 등(2019)은 bipartite GNN에 이를 모방시켜 훨씬 적은 비용으로 이득 대부분을 회수했다. Khalil 등(2016)도 learned ranking으로 같은 수를 둔다. 이득은 새 정보가 아니다 — strong branching은 이미 답을 알고 있었다. 이득은 그 지식이 어디서나 쓸 수 있을 만큼 싸진다는 데 있다. 본 보고서는 같은 구조가 여기에도 적용됨을 주장하고, 그것을 측정한다.

학습이 실패한 곳, 그리고 그 이유. network가 satisfiability를 직접 판정하게 하려는 시도는 기록이 더 약하다. NeuroSAT은 literal-clause graph 위의 message passing으로 satisfiability를 분류하며 작은 random 인스턴스에서 성공하지만, G4SATBench는 크기 transfer에서 15~25%p 정확도 붕괴를 보고한다. 더 근본적으로 Chen 등(2023)은 표준 message-passing GNN이 어떤 feasible/infeasible MIP 쌍을 원리적으로 구분할 수 없음을 증명했다: 그 쌍들은 그러한 network를 규정하는 Weisfeiler-Lehman 위계에서 구분 불가능하다. random node feature가 문헌이 처방한 치료법인데, 우리는 차원 8과 32에서 시험해 AUC 0.4997~0.5003, 즉 우연 수준을 얻었다.

본 보고서가 메우는 공백. 두 문헌 모두 결과를 보고한다 — 이건 됐고 저건 안 됐다. 어느 쪽도 입력이 얼마나 많은 signal을 담는지 규명하지 않으므로, null result를 "정보 없음"과 "정보는 있으나 추출 실패" 사이에 귀속시킬 수 없다. planted-solution 인스턴스에서는 그 귀속이 계산 가능하며, 그것을 수행하는 것이 본 보고서의 방법론적 기여다.

2.4 Amortization: 본 보고서가 채택하는 framing

우리 결과를 조직하는 구분은 조합최적화에서 쓰이기 전부터 있었다. Gershman과 Goodman(2014)은 확률적 추론에 대해 amortized inference를 도입했다: agent가 서로 관련된 많은 query에 답해야 할 때 매번 처음부터 푸는 것은 낭비이며, 유용한 질문은 query 간에 계산을 어떻게 재사용하는가가 된다. Amos(2023)는 같은 아이디어를 amortized optimization으로 전개한다 — "이런 설정에서 학습을 사용해 문제의 해를 예측하며, 유사한 problem instance 간의 공유 구조를 활용한다" — 그리고 variational inference, meta-learning, control, convex optimization에서의 등장을 개관한다.

Millidge(2022)는 그 trade-off를 여기서 가장 유용한 형태로 진술한다. Direct optimization은 당면 instance에 계산을 쓰므로 주어진 compute에 비례해 능력이 커지지만 호출할 때마다 그 비용을 지불한다. Amortized optimization은 문제를 supervised learning으로 바꾼다: inference는 값싼 단일 forward pass지만, 풀린 인스턴스로 이뤄진 비싼 dataset이 선결 조건이고 function approximator의 generalization에 갇힌다. 둘은 경쟁이 아니라 상보적이며, 생산적인 배치는 hybrid다 — AlphaGo처럼 amortized 예측이 direct optimizer를 안내한다.

그 서술의 모든 요소가 아래에 측정된 대응물을 갖는다. 이것이 우리가 단어만 빌리지 않고 framing 자체를 채택하는 이유다. 여기서 direct 방법은 LP-probing이며 건전화되 변수당 LP 한 번으로 비싸다(표 5). amortized 방법은 MarginalNet으로, 단일 forward pass에 약 20배 저렴하다. 비싼 dataset 요구는 우리의 가장 강한 제약으로 나타난다: exact marginal label은 S의 전수열거를 필요로 하는데 이는 $n \approx 60$ 을 넘으면 실패한다 (§ 4.1). generalization 한계는 near-unique 영역에서 18~20%p 잔차로 나타난다 (§ 4.3). 그리고 hybrid 배치가 바로 우리의 설계 원칙이다: network는 branching을 안내하되(오류의 대가는 backtracking), 변수 확정은 건전한 deduction만이 한다(오류의 대가가 정확성일 곳).

본 보고서가 그 문헌에 더하는 것은 빠져 있던 분모다. amortization은 보통 학습된 predictor가 실무적으로 충분히 빠르고 정확함을 보여 정당화되며, 달성 가능한 signal이 얼마인지를 말할 수 있는 경우는 드물다. 그 양이 알려져 있지 않기 때문이다. planted-solution 인스턴스에서는 계산 가능하고, 그것을 계산하면 실패의 해석이 바뀐다: null result를 부족한 모델이 아니라 부재한 ceiling에 귀속시킬 수 있고 — 혹은 본 프로젝트에서 반복적으로 일어났듯 그 반대로도 — 귀속시킬 수 있다.

2.5 본 프로젝트의 이전 사이클

본 보고서가 담고 선 pipeline은 v4~v12 사이클에서 개발됐다. symbolic stage는 probing을 포함한 constraint propagation, LP-relaxation infeasibility rule, affine solution space와 unit hypercube 사이를 교대 투영하는 kernel pump, AHL lattice reduction, 그리고 complete backtracking search다. 학습 stage는 MCTS와 branching을 구동하는 multi-head bipartite GNN(selection, assignment, value, feasibility)이었다.

이전의 두 측정이 여기서 중요하다. 첫째, 인스턴스 단위 귀속 분석에서 학습 stage의 기여는 10×25 에서 7.6%p, 20×50 에서 0.4%p, 40×100 과 60×150 에서 정확히 0이었다 — AHL과 propagation이 전부를 설명했다. 둘째, 예산을 맞춘 연구에서 추가 search compute의 수익이 급격히 체감함을 확인했다(3.8배 시간이 4%p를 샀다). 이는 병목이 모델 용량이 아니라 search 기술에 있음을 가리킨다. § 3 이후는 기여 0인 stage를 폐기하고 학습 컴포넌트를 다른 목적함수 위에 재구축한다.

3. 제안 방법

3.1 개요

방법은 세 부분으로 이뤄진다: 달성 가능한 것을 규명하는 측정 절차 (§ 3.2), planted sample이 아니라 exact posterior에 대해 학습되는 predictor (§ 3.3~3.5), 그리고 predictor를 그 비용 프로파일상 쓸 만한 곳에 투입하는 통합 (§ 3.6~3.7).

3.2 Ceiling 측정

(A,b)가 주어지면 CP-SAT의 all-solutions 모드로 S를 전수열거하고 exact marginal을 계산한다. per-variable 정확도의 Bayes ceiling은 $\text{ceil}(A,b) = (1/n) \cdot \sum_j \max(p_j, 1-p_j)$ 이다. 어떤 결정론적 predictor $f(A,b)$ 도 이를 넘을 수 없다: (A,b)를 공유하는 인스턴스는 f에게 구분 불가능하므로, 변수별로 가능한 최선은 살아남은 해들 중 다수값이다.

틀리기 쉬운 정합성 조건. 열거는 완료됐을 때만 유효하다. CP-SAT는 탐색이 소진되면 OPTIMAL을, 열거할 것이 없으면 INFEASIBLE을, 이미 해를 모은 탐색이 시간제한으로 중단되면 FEASIBLE을 반환한다. FEASIBLE을 완료로 취급하면 잘린 S가 조용히 대입되어 그로부터 계산된 모든 marginal이 편향된다. 우리는 이 버그를 겪었다: n=100에서 |S|가 측정 가능한 것처럼(중앙값 688과 2) 보이게 만들었는데, 실제로는 n=100에서 어떤 열거도 2분 내에 끝나지 않는다. 증상은 평균 소요시간이 시간제한과 정확히 같다는 것이었다. OPTIMAL과 INFEASIBLE만 인정하며, 시간 초과한 인스턴스는 평균에 섞지 않고 폐기한다.

3.3 학습 target: planted sample이 아니라 exact posterior

이전의 모든 사이클은 planted solution에 대해, 즉 $x_j^* \in \{0,1\}$ 을 label로 학습했다. $|S| > 1$ 이면 이는 posterior로부터의 sample이지 posterior가 아니다: 10×25 에서 |S|의 중앙값은 23이고, Bayes-optimal 예측조차 x^* 와 66%만 일치한다. 따라서 x^* 로 학습하는 것은 약 34%의 label noise로 학습하는 것이며, 각 인스턴스를 사실상 한 번만 보므로 그 noise는 평균화될 기회가 없다. 우리는 대신 p_j 에 직접 학습한다. Bernoulli(p_j) target에 대한 cross-entropy이며, 정확히 $p_j^{\hat{}} = p_j$ 에서 최소화된다.

3.4 Conditioning은 reduction과 같다

tree 내부 예측을 값싸게 구현할 수 있게 만드는 단계는 하나의 항등식이다. 변수 집합 F를 값 v로 고정하고 나머지를 K라 하면, $\{x : Ax=b, x_F=v\}$ 는 $\{y \in \{0,1\}^{|K|} : A_K y = b - A_F v\}$ 와 동형이다. 즉 depth-d node에서의 conditional marginal은 정확히 더 작은 BLS instance의 통상적 marginal이다. 하나의 architecture, 하나의 학습 절차, 하나의 label 생성 루틴이 root와 tree 내부 모두를 담당한다; depth-d 상태는 그저 자신의 exact marginal을 가진 더 작은 (A',b') 쌍으로 저장된다.

두 가지 세부가 데이터를 정직하게 유지한다. 상태는 실제 해에서 prefix를 취해 만들므로 모든 학습 상태가 도달 가능하다 — 어떤 해와도 부합하지 않는 prefix는 propagation이 쳐냈을 dead node이지 학습할 가치가 있는 상태가 아니다. 대입 후 전부 0이 된 row는 제거한다: 아무 제약도 부과하지 않으며, SCIP는 빈 제약을 거부한다.

3.5 Architecture

instance는 n개 variable node와 m개 constraint node를 갖고 $a_{ij}=1$ 인 곳마다 edge가 있는 bipartite graph다. variable feature는 LP relaxation 값, column degree, fractionality 3개이고, constraint feature는 tightness, scaled row size, LP residual 3개다. 둘 다 64차원으로 embedding되고 graph attention을 쓰는 message passing 4라운드(variable→constraint, constraint→variable, 각각 LayerNorm+residual)로 정련된다. 2층 head가 변수당 logit 하나를 내어 $p_j^{\hat{}} = \sigma(z_j)$ 를 준다. 이 network를 MarginalNet이라 부른다.

parameter가 m이나 n이 아니라 feature 차원에만 의존하므로 하나의 weight가 임의의 instance 크기에 적용된다 — § 4.7 transfer 실험의 구조적 전제다.

이전 network와의 의도적 대비. 본 프로젝트의 기존 모델은 hidden 256, 8층, 4개 head(selection, assignment, value, feasibility)로 MCTS와 DFS를 담당했다. MarginalNet은 64 폭, 4층, 단일 head다. 약 84배 작는데 § 4.2가 보이듯 이

과제에서 실질적으로 더 정확하다 — multi-head 모델은 LP rounding 수준이었다. head들은 그들이 담당하던, 기여가 0으로 측정된 stage와 함께 폐기했다.

3.6 Predictor의 사용: commitment가 아니라 guidance

MarginalNet은 증명이 아니라 확률을 낸다. 따라서 틀린 답의 대가가 정확성이 아니라 시간인 곳에만 쓴다. 구체적으로 branching은 $|p_j^\wedge - 1/2|$ 가 최대인 변수를 고르고 $[p_j^\wedge \geq 1/2]$ 값을 먼저 시도한다; search는 complete하게 유지되고 실수의 대가는 backtracking이다. 변수의 확정은 건전한 deduction만이 한다 — propagation, 또는 반대값 배정이 relaxation조차 infeasible하게 만들 때 x_j 를 고정하는 LP-probing이다. 이 분업은 § 4.5의 비용-건전성 측정에서 직접 따라 나온다.

3.7 실제 운용 cascade

실제 운용에서 guided search는 단독으로 쓰이지 않는다. pipeline은 AHL lattice reduction을 먼저 돌리고 AHL이 닫지 못한 인스턴스에만 search를 호출한다: propagation → LP rule → kernel pump → AHL/BKZ → guided search. AHL은 guidance 전략에 무관하므로 모든 arm이 정확히 같은 잔여 부분집합을 받는다; 그 위에서 조건부로 보는 것은 유리한 인스턴스를 고르는 것이 아니라 알고리즘을 기술하는 것이다. 우리는 이 구성(AHL on)과 AHL을 제거한 ablation(AHL off)을 모든 크기에서 함께 보고한다. 이전 사이클처럼 ablation만 보고하는 것이 § 4.7에서 바로잡는 오독을 낳았다: AHL을 제거하면 40×100 이 모든 arm에게 풀리지 않게 되는데, 모든 arm에 공통된 붕괴가 학습 arm의 것으로 귀속됐던 것이다.

4. 실험

4.1 인스턴스, 그리고 solution multiplicity 통제

인스턴스는 `gen_hard_feasible(m,n,rng,K)`로 생성한다. 밀도 1/2의 무작위 0/1 행렬 A 에 대해 $K=20$ 개의 후보 해를 삼고 각각 $b=Ax$ 를 계산한 뒤, vertex spread가 최대인 것을 채택한다. 이는 relaxation polytope을 직접 키우며, market-split 인스턴스를 어렵게 만드는 메커니즘이다 (§ 2.1). uniqueness는 강제하지 않는다.

통제 변수로서의 $|S|$. solution multiplicity는 과제의 난이도만이 아니라 성격을 바꾼다: $|S|=1$ 이면 모든 변수가 논리적으로 결정되어 잘 예측한다는 것이 곧 instance를 푸는 것이지만, $|S|=23$ 이면 많은 변수가 진정으로 미결정이고 과제는 부드러운 posterior를 추정하는 것이 된다. 따라서 $|S|$ 를 통제하지 않고 크기를 비교하면 크기와 과제 정체성이 교락된다. 고정된 n 에서 m 이 오르면 $|S|$ 가 줄어들므로, 크기마다 m 을 골라 $|S|$ 를 맞춘다.

크기	m	m/n	$ S $ 중앙값	root ceiling	conditional ceiling
10×25	10	0.40	23	70.4%	87.0%
18×50	18	0.36	19	71.3%	87.2%
21×60	21	0.35	10	—	—

표 1. solution multiplicity를 맞춘 크기들. ceiling이 1%p 이내로 일치하므로 크기만이 변하는 유일한 변수다. 이전에는 같은 비교가 conditional ceiling 87.4% 대 98.6%에서 수행됐다.

그 대가, 그리고 한계. $|S|$ 를 맞추면 m/n 이 변한다($0.40 \rightarrow 0.35$); 고정된 n 에서 둘을 동시에 유지할 수는 없고, 과제 정체성을 결정하는 $|S|$ 를 우선했다. 통제에는 명확한 한계도 있다. $n=60$ 에서 $|S|$ 중앙값은 m 에 따라 급격히 계단을 이룬다 — $m=20$ 에서 42, $m=21$ 에서 12, $m=22$ 에서 2 — 따라서 $m=21$ 이 사용 가능한 최선의 정수이며, 실제 생성된 test set은 중앙값 10에 27/30이 검증됐다. $n=70$ 에서는 적절한 multiplicity를 주는 m 에서 열거가 하나도 완료되지 않았다($m=23$ 에서 0/10). $n=100$ 에서는 실패가 더 근본적이다: CP-SAT가 $m=28,32,36,40$ 에서 심어진 해가 있음에도 20초 내에 해를 하나도 찾지 못한다. 그 크기에서 병목은 guidance 품질이 아니라 해를 하나라도 찾는 것이므로 guidance 비교가 무의미하며 — 이는 이전 연구에서 40×100 의 모든 arm이 8~12%로 붕괴한 이유를 사후적으로 설명한다.

4.2 Bayes ceiling과 모델이 얼마나 근접하는가

10×25 인스턴스에 대해 S 를 전수열거하고(중앙값 $|S|=24$, 각 1초 미만), 분리된 pool의 1,500개 인스턴스로 MarginalNet을 학습한 뒤, 열거가 완료된 held-out 400개에서 평가했다.

Predictor	전체 변수 정확도	top-3	top-5	marginal과의 L_1
Bayes ceiling	69.9%	93.1%	90.5%	0.0000
MarginalNet (hard label)	68.6%	87.4%	85.2%	0.1261
MarginalNet (soft label)	67.7%	87.2%	85.3%	0.0977
기존 multi-head GNN	60.7%	72.0%	69.5%	0.1813
LP relaxation	60.6%	64.5%	64.1%	0.3213

표 2. held-out 10×25 , S 가 전수열거된 400개 인스턴스.

세 가지로 읽힌다. (i) signal은 실재하지만 작다: 69.9%는 우연을 훨씬 상회해 "정보 없음"을 반박하지만, 단독으로 쓰기에는 한참 모자란다 — $0.85^5 \approx 44\%$ 이므로 top-5 정확도조차 5개 변수의 joint commitment를 신뢰할 수 있게 만들지 못한다. (ii) 작은 전용 모델이 그것을 거의 소진한다: MarginalNet은 전체에서 1.9%p, top-3에서 5.7%p 이내인 반면, 기존 multi-head network는 LP rounding과 통계적으로 구별되지 않는다(60.7% 대 60.6%) — 입력 feature인 LP 값을 복사하는 것 이상을 거의 배우지 못했다. 이는 과거의 여러 null을 정보의 부재가 아니라 model/task mismatch로 재해석하게 한다. (iii) label noise 가설은 부분적으로만 맞다: soft label은 posterior 추정을 분명히 개선하지만(L_1 0.1261 $\rightarrow$ 0.0977, -22%) 분류 정확도는 그대로여서, 잡음 label이 과거 실패의 주원인은 아니었다.

4.3 Ceiling sweep: 세 구간, 그리고 크게 이기는 구간의 부재

$n=25$ 를 고정하고 m 을 올리면 $|S|$ 가 줄고 ceiling이 오른다. 각 m 에서 동일한 모델을 학습해 정확도가 따라 오르는지 묻는다.

m	$ S $ 중앙값	unique 비율	ceiling	MarginalNet	격차	LP
10	24	0%	70.5%	68.5%	1.9	61.2%
14	1	76%	96.0%	76.5%	19.6	71.9%
16	1	97%	99.2%	81.2%	17.9	78.4%
20	1	100%	100.0%	99.5%	0.5	99.5%

표 3. $n=25$, m 을 변화시킴. 구간을 가르는 것은 격차 열이다.

$m=10$ — 우리 hard 계열 — 에서는 모델이 ceiling을 포화시키므로 구속 조건은 ceiling 자체이며, 그것이 낮은 이유는 바로 우리가 polytope 크기를 최대화했기 때문이다. $m=14\sim 16$ 에서는 ceiling이 높지만 18~20%p 격차가 열린다: 여기서 높은 ceiling은 해가 사실상 유일하다는 뜻이므로 그것에 도달하는 것은 NP-hard instance를 푸는 것과 같고, 격차는 그 hardness가 가시화된 것이다. $m=20$ 에서는 LP relaxation만으로 99.5%에 이르러 배울 것이 없다. 학습이 큰 격차로 이기는 지점은 존재하지 않는다 — ceiling에 닿을 수 있는 곳에서는 이미 닿아 있고, 높은 곳에서는 계산적으로 닿을 수 없다. 또한 기존 network가 모든 m 에서 LP를 따라간다는 점(60.9 대 61.2; 71.9 대 71.9; 78.5 대 78.4)이 네 구간에 걸쳐 그것이 LP 입력 이상을 배운 적이 없음을 확인해준다.

4.4 Tree 내부: 정보가 실제로 있는 곳

root ceiling 70.5%는 one-shot 예측을 제한하지 search를 제한하지 않는다. § 3.4의 항등식을 써서, 실제 해에서 취한 prefix를 따라 각 depth d 마다 conditional ceiling과 unit propagation이 이미 강제하는 비율을 함께 측정했다.

depth	살아남은 $ S $	conditional ceiling	propagation이 강제	MarginalNet
0	25.2	70.5%	0.0%	67.9%
4	4.1	82.2%	0.4%	73.2%
7	1.9	92.6%	4.1%	80.4%
8	1.6	93.4%	8.2%	85.0%
12	1.1	98.6%	56.7%	97.4%
15	1.0	99.7%	94.6%	—

표 4. depth에 따른 conditional 정보량. $n=25$, 150개 인스턴스, LP-confidence 변수 순서.

ceiling은 가파르게 오른다 — depth 8에서 70.5%→93.4% — 반면 propagation은 남은 변수의 8.2%만 강제한다. tree 내부 상태로 학습하면 root 전용 모델을 depth 6~10 구간에서 5.3~11.5%p 앞서므로, tree 내부 학습 자체가 가치 있음이 확인된다.

우리 자신의 framing에 대한 정정. 우리는 처음에 "학습 창"을 $(\text{ceiling}-0.5) \times (\text{강제되지 않은 비율})$ 로 점수화하며 propagation을 유일한 경쟁자로 취급했다. 그것은 틀렸다: 진짜 경쟁자는 축소된 instance에서 다시 풀 LP relaxation이고, 대입이 polytope을 조이기 때문에 depth에 따라 급격히 강해진다(60.6%→97.2%). 그것을 기준으로 재면 모델의 순이익은 depth 0의 +7.3%p에서 depth 12의 +0.2%p로 감쇠한다. 진정한 잔여는 depth 6~7이며, 거기서 ceiling과의 격차가 가장 크고(10.8~12.2%p) 모델이 여전히 LP를 3.8~4.1%p 앞선다. 방법론적 교훈은 baseline 선정이 결론을 좌우한다는 것이다; 경쟁자 하나를 빠뜨리자 기회가 과대평가됐다.

4.5 잔차의 분해, 그리고 비용 축

depth 6~8에서는 대략 두 개의 해가 살아남으므로, 각 자유 변수는 FORCED(살아남은 해가 모두 일치; $p_i \in \{0,1\}$)이거나 FREE(해들이 불일치)다. ceiling은 정확히 FORCED 변수에서 달성 가능하므로 모든 미달분은 거기 있어야 한다.

depth	FORCED 비율	모델(FORCED)	propagation 탐지	LP-probe 탐지	모델(FREE)
5	61.5%	90.4%	0.9%	62.2%	57.9%
6	72.1%	89.2%	1.6%	68.1%	52.3%
7	83.0%	87.9%	4.1%	75.0%	52.9%
8	89.0%	89.2%	7.5%	79.4%	59.7%

표 5. 잔차 분해. "탐지" 열은 ground-truth FORCED 집합에 대한 recall이다.

격차는 전적으로 FORCED 변수 위에 있다(depth 7에서 $0.83 \times 12.1 \approx 10\%$ p로, 측정된 12.2%p와 부합). 50% 근처의 FREE 성능은 결함이 아니라 올바른 거동이다. 두드러지는 수치는 unit propagation이 논리적으로 결정된 변수의 0.9~7.5%만 탐지한다는 것이다. LP-probing — 반대값을 배정하고 relaxation이 infeasible해지는지 검사 — 은 62~79%를 회수하고, LP integrality와 합치면 82.8~92.0%를 회수한다.

우리는 처음에 이로부터 "classical reasoning이 모델과 대등하므로 학습은 불필요하다"고 결론지었다. 그 결론은 서로 다른 양을 비교한 것이다: probing의 수치는 탐지 recall(발화하면 그것은 증명이고, 그렇지 않으면 기권)이고 모델의 수치는 전 변수에 대한 추측 정확도다. 빠진 축을 넣으면 결론이 정리된다.

depth	MarginalNet 시간	FORCED 정확도	LP-probe 시간	탐지율	속도비
5	1.59 ms	89.9%	36.85 ms	60.5%	23.1×
6	1.61 ms	91.3%	35.30 ms	68.8%	22.0×
7	1.58 ms	88.2%	32.74 ms	77.9%	20.8×
8	1.59 ms	88.5%	30.99 ms	79.8%	19.5×

표 6. instance 전체 판정 1회당 node 비용, 단일 스레드. probing은 변수당 LP를 한 번씩 풀고, network는 단일 forward pass로 전 변수를 답한다.

모델은 약 20배 저렴하면서 적중률은 같거나 높다. 이로써 분업이 확정된다: 틀린 답의 대가가 backtracking인 guidance에서는 network가 엄격히 우월하고, 틀린 답의 대가가 정확성인 건전한 확정에서는 probing이 필수이며 network가 대체할 수 없다. 따라서 학습의 기여는 매 node에서 돌리기에는 너무 비싼 deduction의 amortization이며, 이는 MIP에서 branching imitation이 하는 역할과 구조적으로 동일하다.

4.6 Guided search

네 가지 guidance 전략을 wall-clock 예산을 맞춰 비교한다. branching 품질만 격리하기 위해 AHL을 끈다. 각 arm은 30개 인스턴스를 받는다.

크기 (예산)	guidance	해결율	node 중앙값	시간 중앙값
10×25 (60초)	random	100%	1,514	0.07초
	lp	100%	66	0.12초
	lp-probe	100%	12	0.79초
	model	100%	22	0.08초
20×50 (300초)	random	0%	—	—
	lp	100%	8,876	24.93초
	lp-probe	100%	173	28.14초
	model	100%	1,676	6.37초

표 7. guidance 비교, AHL 비활성화.

trade-off가 명시적이다. lp-probe는 압도적으로 작은 tree를 만들지만(20×50에서 lp 대비 node 51배 적음) wall-clock으로는 가장 느리며, 초당 6 node를 처리하는 반면 lp는 376 node를 처리한다. model은 유용한 지점에 위치한다: lp 대비 node 5.3배 적음을 LP보다 낮은 node당 비용으로 달성해 시간 중앙값이 3.9배 낮다. node 수와 wall-clock이 이 방법들을 정반대 순서로 세운다. 따라서 branching heuristic을 tree 크기만으로 평가하는 통상적

관행은 여기서 결론을 뒤집는다.

4.7 |S| 통제 하의 크기 transfer

전체 요인설계는 네 transfer 조건과 AHL 스위치를 교차한다. 조건 A~C는 10×25에서만 학습한 모델을 쓰고, 조건 D와 n=50 참조는 목표 크기에서 학습한 모델을 써서 transfer 손실을 분리한다.

조건	AHL	arm	해결율	AHL이 해결	탐색 중앙값	node 중앙값
A: 25→25	off	lp / model	100% / 100%	0	0.08 / 0.08초	47 / 23
	on	둘 다	100%	30/30	—	—
B: 25→50	off	lp	100%	0	15.64초	6,208
	off	model	100%	0	2.84초	833
	on	둘 다	100%	24/30	0.03초	—
참조: 50→50	off	model	100%	0	3.42초	1,001
C: 25→60	off	lp	90.0%	0	190.76초	69,015
	off	model	100%	0	47.28초	12,010
	on	lp	93.3%	16/30	잔여 14 중 12	—
	on	model	100%	16/30	잔여 14 중 14	—
D: 50→60	off	model	100%	0	54.75초	14,108
	on	model	100%	17/30	잔여 13 중 13	—

표 8. 조건 A~D × {AHL off, AHL on}, 각 30개 인스턴스, n=60에서 600초 예산.

guidance 품질이 속도에서 해결율로 전환된다. n=50에서 모델은 22/30 인스턴스에서 더 빠르고(중앙값 3.59배) 해결율 차이는 없다. n=60에서는 공통으로 풀린 27개 중 20개에서 더 빠르고(중앙값 2.71배, sign test p=0.019), node를 6.2배 적게 쓰며, 총 시간을 6,227초에서 2,421초로 줄인다 — 게다가 lp가 예산을 소진한 3개 인스턴스를 각각 41초, 230초, 232초에 추가로 푼다. 두 결과는 서로 다른 확신도로 보고한다: 속도 우위는 유의하지만, 해결율 우위는 방향이 완벽하게 일관되지(3-0, lp는 한 개도 이기지 못함) 검정력이 부족하다. 3-0 분할은 McNemar p=0.250이 최선이며 유의성에는 5-0이 필요하다.

Transfer 손실은 측정되지 않는다. 동일한 n=60 test set에서 10×25에서만 학습한 모델과 18×50에서 학습한 모델은 구별되지 않는다: 전자가 정확히 15/30에서 더 빠르고, 속도비 중앙값 0.92배, 총 시간 차이 0.4%(2,924.9초 대 2,914.5초)이며 양쪽 모두 30/30을 푼다. 2.4배 작은 규모에서 학습하는 것이 측정 가능한 대가를 치르지 않는다. root 수준 예측 정확도도 같은 이야기를 한다: 10×25 모델이 10×25, 20×50, 40×100에서 각각 67.8%, 69.5%, 69.4%를 기록하며 LP 대비 우위(+11.0, +7.2, +4.7)를 모든 크기에서 유지한다.

이전 사이클이 이를 실패로 읽은 이유. 이전 사이클은 40×100에서 transfer가 실패했다고 결론지었다. 두 교락이 그 해석을 만들었다: 여기서 바로잡은 |S| 불일치, 그리고 ablation artifact다 — AHL을 제거하면 어떤 arm도 40×100을 풀지 못하는데(모두 8~12%), 이는 순수 DFS가 n=100을 감당하지 못하기 때문이다; 실제 pipeline은 AHL을 써서 거기서 54~62%를 푼다. 모든 arm에 공통된 붕괴가 학습 arm의 것으로 귀속됐던 것이다. 표 8처럼 모든 크기에서 두 AHL 설정을 함께 돌리는 것이 비교를 해석 가능하게 만든다.

운용상의 이득이 어디서 오는가. AHL coverage는 크기에 따라 떨어진다 — n=25에서 30/30, n=50에서 24/30, n=60에서 16/30. 이것이 guidance의 자리를 만든다: n=60에서 14개 인스턴스가 search로 넘어오고, 거기서 model은 14/14를, lp는 12/14를 닫아 end-to-end 해결율을 93.3%에서 100%로 올린다. 이전 사이클들이 학습 컴포넌트의 종단 기여를 0으로 측정한 것은 정확히 lattice reduction이 search가 시작되기 전에 거의 모든 인스턴스를 흡수했기 때문이며, n=60은 그것이 성립하지 않으면서도 search가 여전히 작동하는 첫 크기다.

4.8 Negative result와 그 원인

컴포넌트	결과	규명된 원인
Self-play MCTS policy	9.6→5.9%	open-loop 학습이 사전학습된 search policy를 퇴보시킴
Expert imitation (branching)	8.5→5.5%	fail-first 대 succeed-first 비대칭; 반증에는 도움, 탐색에는 해
AHL restart scheduler 학습	기각	feature rank-AUC 0.54~0.58로 스케줄링하기에 너무 약함
Feasibility classifier (10종)	AUC 0.4997~0.5003	representation이 원리적으로 무정보; near-miss 쌍의 96.8%가 GS profile이 비트 단위로 동일
Random node feature	변화 없음	WL 한계에 대한 문헌의 처방이 여기서는 적용되지 않음
AHL block-size policy	노이즈 수준	기본값 교체가 +14.4%p; 인스턴스별 선택은 추가 이득 없음
CP-SAT parameter policy	signal 없음	config 6종에 대한 oracle이 default와 동일
다중 크기 fine-tuning (109 ep.)	$\Delta=0$	튜닝 대상 stage의 기여가 애초에 0%
AHL permutation policy (RL)	$\Delta=-0.0005$	signal은 존재하나(10개 중 8개가 permutation 의존) 관측 가능한 feature로부터 학습되지 않음
Variable-fixing policy (RL)	완주 불가	k개 결정에 대한 all-or-nothing reward가 credit assignment를 파괴

표 9. 12개 학습 컴포넌트 중 11개가 null. 원인은 추측이 아니라 측정된 것이다.

우리 자신의 해석 두 건도 측정으로 바로잡혔으며, 본 방법론이 잡아내고자 하는 종류의 오류이므로 함께 기록한다: 학습 창을 점수화할 때 propagation을 유일한 경쟁자로 취급한 것 (§ 4.4), 그리고 probing의 탐지 recall과 모델의 추측 정확도를 같은 양인 것처럼 비교한 것 (§ 4.5).

5. 논의

5.1 측정이 뒷받침하는 것

방어 가능한 주장은 좁고, 프로젝트가 처음 세웠던 것보다 유용하다고 본다:

학습은 classical reasoning이 닿을 수 없는 정보를 공급하지 않는다. 학습은 매 node에서 돌리기에는 너무 비싼 deduction을 쓸 수 있을 만큼 싸게 만들며, 이 능력은 solution multiplicity가 통제되면 instance 크기를 넘어 측정 가능한 손실 없이 전이된다.

각 절이 측정됐다. "새 정보를 공급하지 않는다": 잔차가 전적으로 논리적으로 FORCED된 변수 위에 있고, 그중 82~92%를 classical probing이 회수한다 (§ 4.5). "매 node에서 돌리기에는 너무 비싸다": LP-probing이 node당 31~37ms인 반면 network는 1.6ms다(표 6). "측정 가능한 손실 없이 전이된다": 15/30과 총 시간 차 0.4%다 (§ 4.7).

더 약한 주장 — 학습이 classical 방법이 놓치는 구조를 찾아낸다 — 은 우리 자신의 데이터와 모순되며 해서는 안 된다.

Amortization 문헌의 예측과 대조. Millidge(2022)의 trade-off 서술 (§ 2.4)을 기준으로 읽으면, 우리 결과는 그 서술이 있어야 한다고 말하는 자리에 정확히 놓인다. 비용까지 포함해서 그렇다. 예측된 이득 — inference가 값싼 단일 forward pass로 축소됨 — 은 31~37ms 대비 1.6ms로 실현되고, 예측된 hybrid 이점은 cascade에서 실현되어 guidance가 변수 확정을 한 번도 말지 않으면서 end-to-end 해결율을 93.3%에서 100%로 올린다. 예측된 비용도 똑같이 보인다. 풀린 인스턴스로 이뤄진 비싼 corpus 요구는 우리의 구속 조건이다. exact marginal이 전수열거를 요구하고 열거는 $n \approx 60$ 을 넘으면 실패하기 때문이다. approximator의 generalization이 부과하는 한계는 $m=14\sim 16$ 에서의 18~20%p 잔차이며, 거기서는 ceiling이 높지만 도달이 NP-hard다.

그 framing이 예상하지 못한 결과가 하나 있어 따로 적어둔다. amortization은 보통 효율을 근거로 옹호되고 정확도는 모방 대상 expert 대비 열화한다고 가정되는데, 여기서는 amortized predictor가 FORCED 집합에서 LP-probing보다 더 싸면서 동시에 더 정확하다(탐지 61~80%에 대해 88~91%, 표 6). 이유는 둘이 서로 다른 질문에 답하기 때문이다: probing은 자신의 검사가 발화하지 않는 곳에서 기권하는 반면 network는 어디서나 답을 낸다. 어차피 어떤 변수든 분기해야 하므로 기권에 가치가 없는 guidance에서는, 부분집합을 정확히 증명하는 것보다 어디서나 적당한 정확도로 답하는 것이 우월하다. 따라서 적절한 비교 기준은 expert 대비 정확도가 아니라 예측의 소비자가 사용할 수 있는

정확도이며, 그 소비자는 branching rule이다.

5.2 한계

Ceiling이 낮고, 그것을 낮춘 것은 우리다. vertex spread 최대화는 이 인스턴스를 CP-SAT에게 어렵게 만드는 요인이자 root ceiling을 69.9%로 끌어내리는 요인이다. 더 쉬운 계열은 더 많은 signal을 제공하겠지만 목표 영역이 아니다.

유용한 크기 창이 좁다. 그 아래($n \leq 25$)에서는 lattice reduction이 전부를 담아 guidance가 무의미하고, 그 위($n=100$)에서는 search가 아예 실패해 CP-SAT조차 20초 내에 해를 하나도 찾지 못한다. 우리의 시연은 $n=60$ 에 있으며, 창이 더 확장된다는 것은 보이지 못했다.

열거는 평가만이 아니라 방법 자체를 제한한다. exact marginal label은 S의 열거를 요구하는데 이 계열에서는 $n \approx 60$ 을 넘으면 실패한다. transfer는 열거 가능한 크기에서 학습한 모델을 더 큰 크기에 적용할 수 있다는 뜻이지, 거기서 학습 데이터를 만들 수 있다는 뜻이 아니다.

통계적 검정력. $n=60$ 에서의 해결율 우위는 방향상 3-0이지만 $p=0.250$ 이며, 속도 우위($p=0.019$)만이 유의하다. 전자를 확정하려면 더 큰 test set이 필요하다.

|S| 통제는 근사적이다. 중앙값이 23, 19, 10이며, $n=60$ 에서는 연속한 정수 m 에 대해 multiplicity가 $42 \rightarrow 12 \rightarrow 2$ 로 계단을 이루므로 더 정밀한 matching이 불가능하다. test 인스턴스 30개 중 3개는 |S|가 미검증이다. |S|를 맞추는 것은 m/n 을 0.40에서 0.35로 이동시키기도 한다.

단일 응용 영역. 모든 결과가 하나의 응용에서 나온 하나의 인스턴스 계열에 관한 것이다. amortization framing이 다른 constraint 계열로 전이되는지는 검증되지 않았다.

5.3 향후 연구

측정에서 따라 나오는 세 방향이다. (i) 열거를 넘어서는 근사 marginal: p_j 를 sampling(solution sampling 또는 belief propagation)으로 쓸 만한 충실도로 추정할 수 있다면, 열거가 실패하는 곳에서도 학습 데이터를 만들 수 있다. 이는 Millidge(2022)가 amortization의 특징적 대가로 지목한 dataset 비용 항이며, 이를 완화하는 것이 방법의 도달 범위를 가장 크게 넓히는 단일 변화다. (ii) LP-probing을 직접 모방하는 학습: 표 6은 network가 모방하도록 학습되지 않았는데도 이미 1/20 비용으로 probing의 적중률에 필적함을 보인다; probing 결정으로부터의 명시적 distillation이 자연스러운 다음 단계이며, 정확히 learn2branch의 조리법이다. (iii) pipeline의 propagator 교체: unit propagation이 FORCED 변수의 8% 미만을 탐지하는 반면 LP-probing이 62~79%를 탐지한다는 측정은 학습과 무관하게 즉시 적용 가능한 개선이다.

6. 결론

binary linear system에 대한 11개 사이클의 null result는 입력이 학습 가능한 signal을 담지 않는다는 증거로 읽혀왔다. 이 인스턴스들은 planted solution을 갖기 때문에 그 해석은 검증 가능했고, 틀렸다. solution set을 열거하면 exact posterior와 따라서 exact Bayes ceiling이 나온다: hard 계열에서 per-variable 69.9%, 가장 확신하는 변수 3개에서 93.1%다. 작은 단일 head network가 1.9%p 이내에 도달하는 반면 기존 multi-head network는 LP rounding 수준에 머물렀다 — 따라서 이전의 여러 null은 정보의 부재가 아니라 model과 task의 mismatch를 반영한 것이다.

ceiling을 통제 변수로 바꾸면 그럼에도 signal로 이득을 보기 어려운 이유가 드러난다: ceiling이 낮은 곳에서는 이미 도달했고, 높은 곳에서는 도달이 NP-hard이며, 더 높은 곳에서는 LP relaxation으로 충분하다. 따라서 생산적인 질문은 network가 얼마나 아는가가 아니라 얼마나 싸게 아는가이다. search tree 내부에서 conditional ceiling은 depth 8까지 93.4%에 이르고, 잔차는 전적으로 prefix가 논리적으로 강제하는 변수 위에 있으며, classical LP-probing이 변수당 LP 한 번으로 그 대부분을 회수하는 반면 network는 단일 1.6ms pass로 전 변수를 답한다 — 같거나 더 나은 정확도에서 19~23배의 비용 우위다. branching guidance로 쓰면 21×60 에서 중앙값 2.71배 가속($p=0.019$), node 6.2배 감소, 해결율 90.0% 대비 100%를 준다. lattice reduction이 흡수하는 비율이 $n=25$ 의 30/30에서 $n=60$ 의 16/30으로 떨어지는 실제 cascade에서는 guidance가 end-to-end 해결율을 93.3%에서 100%로 올린다. solution multiplicity를 크기 간에 맞추면 10×25 에서만 학습한 모델이 목표 크기에서 학습한 모델과 통계적으로 구별되지

않으며, 이전의 "transfer 실패"는 ablation artifact로 설명된다.

여기서 학습이 맡는 역할은 비싼 deduction의 amortization이며 — mixed-integer programming의 branching imitation에서 맡는 바로 그 역할이다 — classical reasoning이 닿을 수 없는 구조의 발견이 아니다. 이는 본 프로젝트가 세우려 했던 것보다 좁은 주장이지만 측정이 뒷받침하는 주장이며, 이를 규명하는 측정 절차는 planted solution을 갖는 임의의 조합 문제 계열에 적용된다.

재현성

코드는 proposed_src/ 아래에 사이클별로 정리되어 있으며, 각 폴더의 ROLES.txt가 모든 파일과 그 사이클의 결과를 기술한다; util/은 사이클 간 공용 모듈을 담는다. 스크립트는 각자의 폴더에서 실행한다. 전체 사이클은 **cd proposed_src/v22_transfer && bash run_v22_cd.sh**로 재현되며, 이는 |S|를 맞춘 test set을 만들고 조건 C와 D를 두 AHL 설정 모듈에서 실행한 뒤 교차 비교 리포트를 출력한다. 저장소는 코드와 문서만 추적한다: 가중치와 인스턴스 데이터는 저장소를 가볍게 유지하기 위해 제외했으며, 모든 산출물은 docs/version.md 부록 A에 명시된 스크립트로 재생성된다. 생성기는 모두 seed가 고정되어 있으므로 재생성된 데이터는 본 보고서의 수치를 만든 것과 일치한다. guidance 모델은 runs/v22/model_n25/conditional.pt(96KB)와 runs/v22/model_n50/conditional.pt이고, 기존 multi-head network는 runs/best.pth(8.1MB)다.

참고문헌

- Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.
- Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.
- Amos, B. (2023). Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732.
- Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421.
- Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *ICLR*.
- Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *IPCO*, pages 284–293.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *NeurIPS*.
- Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In *Proceedings of the 36th Annual Meeting of the Cognitive Science Society*, pages 517–522.
- Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In *AAAI*, pages 724–731.
- Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.
- Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *TMLR*.
- Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.
- Millidge, B. (2022). Deconfusing direct vs amortized optimization. *비심사 기술 노트*, <https://www.beren.io/2022-09-25-Deconfusing-direct-vs-amortized-optimization/>
- Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., and Malik, S. (2001). Chaff: Engineering an efficient SAT solver. In *DAC*, pages 530–535.
- Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.
- Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1–3):181–199.

- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In ICLR.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In ICLR.
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful are graph neural networks? In ICLR.